

while keeping it simple. Additional functionality can be layered on top of the API set.

Calls in the specification serve as examples of functionality. These are not mapped to an existing implementation.

MPI is a message-passing library interface specification. Unlike OpenMP, it can be used on either shared or distributed memory architecture. However, MPI requires more changes to source codes than OpenMP, but it does not require any special compilers.

OpenMP API uses fork-join model of parallel execution. Multiple threads of execution perform tasks defined implicitly or explicitly by OpenMP directives. It is intended to support programs that will execute correctly both as parallel programs (multiple threads of execution and a full OpenMP support library) and as sequential programs (directives ignored and a simple OpenMP stubs library).

However, it can develop a program that executes correctly as a parallel program but not as a sequential one. Or it can develop a program that produces different results when executed as a parallel program compared to when it is executed as a sequential program.

Further, using different numbers of threads may result in different numeric results, because of changes in the association of numeric operations. For example, a serial addition reduction may have a different pattern of addition associations than a parallel reduction. Different associations may change the results of floating-point addition.

When an OpenMP program begins, an implicit target data region for each device surrounds the whole program. Each device has its data environment defined by its implicit target data region. Any declared target directives that accept data-mapping attribute clauses determine how an original variable in a data environment is mapped to a corresponding variable in a device data environment.

When an original variable is mapped to a device data environment and the associated corresponding variable is not present in it, a new corresponding variable (of same type

and size as original variable) is created in the device data environment. Initial value of the new corresponding variable is determined by clauses and data environment of the encountering thread.

CORBA, or common object request broker architecture, is meant for inter-object communication between computers. It provides high abstraction and several other services. Attempts have been made to use CORBA on embedded systems, and some of its parts have been implemented in VHSIC hardware description language (VHDL). For hardware, CORBA is, however, too heavy.

Communicating entities

MCAPI specification is used for both API and communications semantic. It does not define which link management, device model or wire protocol is used underneath. As such, by defining a standard API, it provides source code compatibility for the application code to be ported from one environment to another (for example, from PC to MPSoC). Implementation of MCAPI also hides the differences in memory architectures. MCAPI communication is based on node and end-point abstraction.

MCAPI node is a logical concept that can be denoted to many entities, including process, thread, hardware accelerator or processor core. Nodes are always unique and statically defined at the time of design. Each node can have multiple end-points that are socket-like termination points. For example, an encryption node could be implemented as a single thread receiving plain text from one end-point, and sending encrypted data via another.

End-points are defined with a tuple $\langle \text{node_id}, \text{endpoint_id} \rangle$, and can be created at run time. MCAPI channels can be dynamically created between pairs of end-points. However, channel type and direction cannot be changed without deleting and recreating the channel. Multi-cast and broadcast is not supported.

Communication types

MCAPI defines three types of communication as explained next:

1. Messages: connection-less datagrams
2. Packet channels: connection-oriented, uni-directional, FIFO packet streams
3. Scalar channels: connection-oriented single-word, uni-directional, FIFO packet streams

Messages are flexible, and useful when senders and receivers dynamically change and communicate infrequently. These are commonly used for synchronisation and initialisation.

Packet and scalar channels provide lightweight socket-like stream communication mechanisms for senders and receivers with static communication graphs. In multi-core, these channel APIs provide a low-overhead ASIC-like, uni-directional FIFO communications capability.

MCAPI messages transmit data between end-points without first establishing a connection. Memory buffers on sender and receiver sides are provided by user application. These messages may be sent with different priorities.

Multi-processor system-on-chip

An MPSoC platform (Fig. 2) includes two processing elements (PEs), HIBInetwork as well as direct memory access (DMA) interfaces for IP blocks (HIBI PE DMA) and external memory (HIBI MEM DMA). Processors have local, private instruction and data memories. Inter-processor communication is implemented using message-passing.

The architecture is in an FPGA on a development board connected via PCIe to a PC, or embedded CPU board. The platform is synthesised to Altera's Arria II GX FPGA development board at 100MHz.

A regular PC (2.4GHz, 1024MB) running Debian GNU/Linux 5.0.6 is used for application development and FPGA synthesis, to demonstrate that an application code can easily run either on PC or NIOS. This is because MCAPI takes care of inter-processor communication, for PCIe, HIBI or any other. Hence, applications can be developed in a workstation environment that allows simpler debugging. Then, application code